

EasySave - Guide des Tests Unitaires

Champ	Valeur
Projet de tests	EasySave.Tests
Cible	net10.0
Framework	xUnit + Moq + coverlet.collector
Derniere mise a jour	25/02/2026

1. Objectif

Ce guide explique:

- les tests existants
- comment les lancer
- comment mesurer la couverture
- comment ajouter des tests de qualite homogène

2. Perimetre actuel

Le projet `EasySave.Tests` couvre principalement la logique du coeur applicatif:

Fichier de test	Cible fonctionnelle
<code>BackupJobTests.cs</code>	Validation du modele <code>BackupJob</code>
<code>BackupFileFilterTests.cs</code>	Filtrage full/differential + extensions prioritaires
<code>BackupStateTests.cs</code>	Calcul progression + transitions de statuts
<code>BackupStateServiceTests.cs</code>	Initialisation, ecriture de progression, finalisation
<code>BackupValidationServiceTests.cs</code>	Verification source + blocage logiciel metier

Ce qui n'est pas couvert actuellement:

- rendu GUI Avalonia
- orchestration complete multi-jobs (integration)
- pipeline end-to-end avec LogServer/Docker

3. Execution des tests

Depuis la racine du depot:

```
dotnet test EasySave.slnx
```

Execution du seul projet de tests:

```
dotnet test EasySave.Tests/EasySave.Tests.csproj
```

4. Execution avec couverture

```
dotnet test EasySave.Tests/EasySave.Tests.csproj --collect:"XPlat Code Coverage"
```

Sortie coverage:

- `EasySave.Tests/TestResults/<run-id>/coverage.cobertura.xml`

5. Filtres utiles

Par classe:

```
dotnet test EasySave.Tests/EasySave.Tests.csproj --filter "FullyQualifiedName~BackupFil
```

Par nom de methode:

```
dotnet test EasySave.Tests/EasySave.Tests.csproj --filter "Name~IsSourceDirectoryMissin
```

6. Standard de qualite pour nouveaux tests

6.1 Convention de nommage

Format recommande:

- `Methode_Contexte_ResultatAttendu`

Exemple:

- `ExecuteBackupAsync_WhenPaused_WritesPausedState`

6.2 Structure recommandée

```
[Fact]
public void Methode_Contexte_ResultatAttendu()
{
    // Arrange
    // Act
    // Assert
}
```

6.3 Regles pratiques

- un comportement principal par test
- assertions précises (éviter les assertions vagues)
- tests déterministes (pas de dépendance à l'heure système sans contrôle)
- isolation des IO via dossiers temporaires
- dépendances extérieures mockées via interfaces

7. Patterns utilisés dans le projet

- Moq pour `IFileService`, `IProgressWriter`, `IBusinessSoftwareMonitor`, `IBackupLoggerService`
- dossiers/fichiers temporaires pour scénarios filesystem
- vérification des appels de log/progression (`Verify(...)`)

8. Roadmap de tests recommandées

Priorités hautes:

1. `JobExecutionService` : parallélisme, pause/reprise, cancel.
2. `EncryptionService` : fallback CryptoSoft absent, codes d'erreur.
3. `SharedExecutionContext` : coordination priorités + gros fichiers.
4. `UserConfigManager` : robustesse sur JSON invalide.
5. `JobStateFileReader` : comportement sur fichier partiel/corrompu.

Priorités moyennes:

1. `DockerLoggerService` avec faux serveur TCP.
2. tests integration "small scenario" (jobs + state + logs).

9. Contrôle avant merge

Checklist minimale:

1. `dotnet test` passe sans échec.

2. Les nouveaux tests couvrent le comportement ajoute.
3. Les assertions valident les effets observables (etat, logs, retours).
4. Aucun test fragile lie a l'environnement local.

10. Conseils CI

- executer `dotnet test EasySave.slnx` sur chaque PR
- publier le rapport de couverture en artifact
- bloquer merge si tests rouges

11. References

- Full guide EN: `docs/user-guide/en/EasySave_User_Guide_Full.md`
- Guide complet FR: `docs/user-guide/fr/Guide_Utilisateur_EasySave_Complet.md`
- Guide debug: `docs/debug/DEBUG_GUIDE.md`